

Impact of Latency on Applications' Performance

Rossen Dimitrov and Anthony Skjellum
{rossen, tony}@mpi-softtech.com

MPI Software Technology, Inc.
101 S. Lafayette Str., Suite 33
Starkville, MS 39759
Tel.: (662) 320-4300
<http://www.mpi-softtech.com>

Abstract

This paper investigates the impact of point-to-point latency on applications' performance on clusters of workstations interconnected with high-speed networks. At present, clusters are often evaluated through comparison of point-to-point latency and bandwidth obtained by ping-pong tests. This paper shows that this approach to performance evaluation of clusters has limited validity and that latency has minimal impact on a large group of applications that use medium- to coarse-grain data-parallel algorithms. Message-passing systems with low latency often use polling for message completion, which leads to tight synchronization between the communicating processes and high CPU overhead. Systems with asynchronous message completion have higher point-to-point latency for short messages but offer a number of high-performance mechanisms such as overlapping of computation and communication, independent message progress, efficient collective algorithms, asynchronous processing of communicating nodes, and exploitation of temporal locality. These mechanisms can be effectively implemented on current high-speed networks with intelligent interface controllers capable of bus-master transfers on host peripheral busses. Although message-passing systems that use asynchronous completion notification have higher point-to-point latency than systems with polling, they can offer opportunities for performance gains with far greater overall impact.

1 Introduction

In recent years, clusters of workstations have become the fastest growing choice for building parallel platforms. The success of clusters has been mainly facilitated by the rapid advancement of microprocessor technologies and high-speed network interconnects, such as Myrinet [2] and Giganet [6]. The increased accessibility and low cost of these technologies are also important factors for the wide acceptance of clusters among users. The recently

introduced Virtual Interface (VI) Architecture [3] standard for high-speed interconnects makes another step toward facilitating high-performance parallel processing on clusters. This standard allows for zero-copy, low latency message transfers between networked computers. In addition, VI Architecture network interface controllers can perform independent message transmission, which reduces significantly the CPU usage for communication and increases the sustainable peak bandwidth. The features of modern high-speed networks combined with the increasing floating-point performance of microprocessors make clusters a low cost alternative to supercomputers with comparable performance.

The characteristics of VI Architecture networks are a close match to the concepts of the Message-Passing Interface (MPI) [9]. These networks provide reliable, ordered transports, which is the major low-level service required by the MPI communication subsystem. Such transports eliminate the need for high-level reliability layers and simplify MPI implementations. MPI/Pro [5] is the first MPI implementation that provides specifically targeted optimizations for VI Architecture. Among the VI networks that MPI/Pro supports are Giganet cLAN and Compaq Servernet. In addition, MPI/Pro supports Myrinet, which is not VI Architecture compliant but offers similar advanced mechanisms for efficient communication.

In this paper, a background of the growing use of clusters for high-performance parallel computing is presented. Then, widely used models and metrics for parallel processing are reviewed. Synchronous and asynchronous modes for message completion are discussed, as are mechanisms used for implementation of these modes and their impact on point-to-point latency and bandwidth, as well as on CPU overhead. Then, results from experiments on a cluster with MPI using alternatively both modes of completion are presented. Tests from point-to-point latency and bandwidth measurements and from the NAS Parallel Benchmarks [1] are shown. Finally, conclusions about

the impact of latency and message completion modes on applications performance are drawn.

2 Background

At present, two major categories of organizations build clusters for parallel processing. The first group includes national laboratories and large businesses. They have had long experience in parallel computing on massively parallel processors (MPPs). Now, these organizations view clusters as a low-cost path for upgrading their processing facilities. The second group of organizations, such as academic labs and small businesses, has been able to experience the benefits of parallel processing for first time by the use of clusters. Even though the two categories have different background, both of them meet the same challenge of adjusting to a new environment.

In general, clusters are just one representative among parallel platforms. However, clusters have a number of characteristics that need to be adequately addressed for full understanding of the processes and interactions between hardware and software as well as their impact on performance and scalability. Clusters differ substantially from MPP systems in several aspects. Some of these aspects are interconnecting topologies, available bandwidth per node, type of operating systems, degree of synchrony, and communication protocol stacks. Consequently, performance evaluation of clusters requires a different approach than the evaluation of MPP systems. In addition, the organizations that are new to parallel processing are challenged by the transition from sequential to parallel systems.

With the fast growth of cluster-based solutions and the rapid increase of organizations that enter for first time the area of parallel processing, a somewhat simplistic approach to performance evaluation can be observed. The evaluation procedure is often reduced to measurements of point-to-point latency and bandwidth with ping-pong tests. Since the asymptotic peak bandwidth of most high-speed networks is approximately the same and the message-passing middleware usually reaches this peak, the difference between the various cluster solutions is expressed only by the difference in latency. The results of these point-to-point tests are then extrapolated to the entire cluster. As a result, conclusions about the performance and scalability of the entire parallel system are made based only on the latency numbers.

There are several reasons for this simplistic approach. First, the organizations with experience in parallel computing are used to certain type of behavior and performance characteristics of MPP systems and MPI implementations for these systems. Throughout the years, a large number of legacy applications have been created for MPP environments. These

applications reflect the specific characteristics of the MPP systems. Now, while moving to clusters, these organizations naturally use their long experience in parallel processing not always realizing that clusters have different characteristics than MPP. Well-established procedures and test suites for performance evaluation are being reapplied to clusters. Second, the organizations that are just building their first clusters often do not have enough knowledge in parallel algorithms and evaluation of parallel platforms. Because ping-pong tests are easy to carry out and understand, they have attracted the attention of the second category of organizations. The third factor for the emphasis on latency can be traced to some of the models for parallel computation, such as BSP [10] and LogP [4]. These models use point-to-point latency and bandwidth for expressing the overall performance of a given algorithm on abstract parallel platforms. Each specific parallel platform presents the actual values for the parameters used by the models, which are then used for predicting the performance on this system.

This paper shows that under equivalent hardware and software conditions, a noticeable difference in point-to-point latency does not yield a corresponding increase of overall performance. The goal is to demonstrate that for a large group of parallel applications the overall impact of latency optimizations may be minimal or even not existent. Since the lowest possible latency is often achieved through a specifically organized MPI middleware, a number of performance optimizations with potentially far greater impact on performance can be missed. Some of these mechanisms are minimizing CPU overhead, overlapping of computation and communication, independent message progress, and asynchronous processing.

The objective of this paper is to show that by using software and hardware architectures that might lead to increase of point-to-point latency, potential for significant performance improvements can be created. Also, this paper shows that these potentials can be achieved at little or no expense to applications and algorithms that do not take use these mechanisms, which is an important attribute of each technique used for improving performance in general.

3 Models for Parallel Computing

Models for parallel computing aim to describe the interaction between software and hardware on parallel platforms through high-level abstractions. The goal of these models is to offer a representation of parallel algorithms independent of the specific characteristics of a given architecture. This goal is usually achieved through expressing the performance of a parallel algorithm as a function of a number of basic parameters, which are empirically obtainable. Once

the abstract performance expression for an algorithm is derived, experimental tests on different platforms can be executed in order to obtain the values of these basic parameters. Then, their values are substituted in the expression and an estimate about the overall performance of the algorithm on the specific platform is obtained.

Two of the most widely accepted models for parallel processing on message-passing systems are BSP and LogP. BSP views the execution of a parallel program as a series of super-steps [10]. In each superstep, processors perform local computation followed by communication. The superstep ends with global synchronization. The time for each superstep is expressed as a function of two parameters – l and g , which can be viewed as message passing latency and bandwidth. The total execution time of the algorithms is obtained by the sum of the superstep times.

The LogP model goes a step further than BSP in describing the communication processes. Except parameterization of the network medium, LogP introduces a description of processor activities associated with communication. This is achieved through the overhead o parameter. The model uses four parameters for deriving performance expressions of algorithms – L and g representing latency and bandwidth similarly to BSP, and o and P representing processor overhead and number of processors involved in the computation. LogP provides for a more precise description of the communication processes and allows for asynchronous processing by eliminating the global synchronization phase of BSP. However, the LogP model does not account for overlapping of computation and communication, which can be a significant source of performance gains. As opposed to LogP, BSP allows for overlapping if the algorithm can schedule computation and communication activities simultaneously.

4 Parallel Performance Metrics

Metrics are necessary tools for evaluating any physical process or phenomenon. Similarly, evaluating performance of parallel algorithms and platforms relies on metrics. Parallel performance metrics and evaluation methodologies have been subject to numerous research efforts [8, 11]. Metrics can be effectively divided into two groups – metrics that measure point-to-point performance and metrics that view the parallel system as a whole. In this paper, these two groups of metrics will be referred to as point-to-point and collective metrics.

Collective metrics are based on applications execution time and reflect the contribution of each processor. These metrics emphasize both performance and scalability, which makes them a powerful tool for studying the complex processes in a parallel system.

Throughout the years of theory and practice in parallel processing, several collective metrics have received the widest acknowledgement. Among these are the following:

- Parallel execution time: T_p ,
- Speedup: T_1/T_p ,
- Efficiency: T_1/pT_p ,
- Performance: $MFLOP/sec$, and
- Cost-efficiency: $MFLOP/sec/\$$,

where T_1 is the execution of the best known serial algorithm and p is the number of processors. The two most widely used point-to-point metrics are latency and bandwidth. These metrics are intended to reflect the entire communication subsystem including the physical and data-link layers of the interconnection network, the host operating system, the network drivers, and the message-passing middleware. Although point-to-point metrics reflect the communication performance of a single link, it is frequently assumed that the performance parameters of this link are preserved across the parallel system during the execution of applications. This assumption is too optimistic for a large number of practical systems because it ignores important scalability factors such as network contention, bisection bandwidth, as well as communication and application software architectures. It is not the objective of this paper to study the scaling properties of point-to-point metrics. Rather, the paper attempts to analyze specifically the affect of message-passing latency of a cluster-based parallel system on the overall applications performance.

Traditionally, point-to-point measurements have been performed by ping-pong tests. This type of tests with a little variation has also been accepted in the area of cluster computing as the predominant benchmark for latency and bandwidth. Because the asymptotic peak bandwidth of high-speed networks is usually similar - upper bounded by the I/O bus throughput, often the performance characteristics of the entire cluster are extrapolated solely from the ping-pong latency numbers; the lower the latency is, the higher the performance of the cluster is. This approach has limited validity for several reasons:

- Latency affects only the exchange of short messages. Typical parallel applications that are based on the message-passing paradigm use medium to coarse grain data parallel algorithms and the messages that they generate are in the range of tens of kilobytes to megabytes. Latency has a minimal impact on the transmission of such messages.
- The ping-pong tests do not show any insights about the costs paid by the system and applications software for achieving the lowest point-to-point latency. The message-passing middleware, such as

MPI, inevitably makes architectural compromises in order to reduce latency to a minimum. These compromises are not revealed by the ping-pong test, so exaggerating the impact of point-to-point latency disregards such sources of performance as overlapping of computation and communication, independent message progress, optimized collective algorithms, and low CPU overhead. These mechanisms are usually sacrificed by MPI implementations that aim solely at the lowest ping-pong latency.

- Although parallel computation models use latency and bandwidth for deriving expressions for the overall applications performance, it is hardly proven that the ping-pong tests actually reflect the latency and bandwidth parameters that are meant in these parallel models – the ones that will be experienced by the applications. Also, it is not obvious that ping-pong latency will scale across the entire system under heavy computation and communication loads. Therefore, in addition to ping-pong tests between two nodes, further and more elaborate collective tests are necessary.

There are a number of alternatives for measuring latency and bandwidth. Obviously, one of them is the ping-pong test, to which we have already alluded. This test subjects the system to highly synchronized and regular communication traffic. Communication patterns of real parallel applications are often asynchronous and they do not follow this ordered scheme according to which nodes send a message with a certain size and then wait for another message with the same size to arrive back. The ping-pong traffic pattern can be viewed as one of the extreme boundary points in a space describing the traffic patterns of parallel application. The other extreme point in this space is when one of the two communicating nodes sends consequently a large number of messages in one direction and the other node only occasionally returns a message back. The test for measuring latency and bandwidth with this pattern is the called streaming test. Using ping-pong and streaming tests, one can measure latency and bandwidth at the end points of the traffic pattern space and obtain a more objective picture of the point-to-point performance characteristics of the parallel system.

Ping-pong latency is computed as a half of the message roundtrip time (RTT), while the streaming latency, referred to also as one-way-latency (OWL) is computed by measuring the time for sending n messages in one direction followed by one message in the opposite direction divided by $n + 1$. Similarly, bandwidth is computed as the ratio of message size and either RTT or OWL.

- $L_{rtt} = RTT/2$
- $L_{owl} = (T_n + T_1) / (n + 1)$
- $B_{rtt} = msg_size / L_{rtt}$
- $B_{owl} = msg_size / L_{owl}$

L_{rtt} , L_{owl} and B_{rtt} are used for evaluating point-to-point performance.

5 Message Completion Modes

Communication between processes in message-passing systems requires explicit participation of both the sender and the receiver. In order to free data buffers for reuse, the message-passing middleware has to ensure completion of the transmission operation. There are two forms of message completion – synchronous and asynchronous. The synchronous form is usually implemented through the use of polling. Depending on the underlying communication layer, whether kernel-based such as TCP/IP or user level networking with operating system bypass, the completion can be performed by polling on a system call or a flag in memory that is signaled by the network controller through a system bus transaction.

Polling requires that user processes make frequent calls to the message-passing middleware so that timely progress of messages can be guaranteed. Polling eliminates all unnecessary overhead from message completion, which is a prerequisite for achieving the lowest latency. However, the host CPU is used for busy waiting and cannot perform any other computation while polling for message completion. This eliminates the possibility of computation and communication overlapping and true asynchronous progress. Applications with highly synchronized execution pattern that exchange predominately small messages benefit most from polling. However, if the execution lines of the communicating processes are asynchronous, a significant portion of the CPU time is wasted on polling for completion. Evidently, because of its high degree of synchrony ping-pong tests favor systems with polling mode of completion. One typical representative of message-passing middleware that relies on polling for completion is MPICH [7].

The asynchronous method for message completion is implemented through callback handlers or progress threads that block until a communication event is completed. Since this method introduces an extra context switch, it increases the communication overhead. Because of its traffic pattern, the ping-pong test mechanically adds this extra overhead to the measured latency. As a result, the asynchronous method for completion shows higher latency than the polling method. On the other hand, asynchronous completion reduces CPU overhead – the time spent by the CPU for communication. The CPU is interrupted only when a communication event associated with a message completion is signaled. Thus, the CPU wastes minimum time on synchronization and can spend more cycles on useful computation. The intelligent bus-master capable controllers of modern high-speed networks offer a number of mechanisms

that further facilitate the asynchronous mode. These controllers perform a significant portion of the message transmission. They can access user data buffers across the peripheral bus and send/receive frames to/from the network without any involvement of the host CPU. The CPU is only interrupted to propagate the completion event to the user process.

Another important advantage of the asynchronous mode is that it provides for an optimal architecture of the message-passing middleware when supporting multiple simultaneous communication devices, such as TCP/IP, shared memory, and high-speed network device. This architecture allows for independent progress of messages on different devices without causing an interference of slow devices into the operation of fast devices

Below, this paper compares two message-passing systems with equivalent characteristics with the exception of their message completion modes. The goal is to investigate how the difference in latency caused by the completion mode affects applications, in this case represented by the NAS benchmarks. The hypothesis is that for a large number of parallel applications latency has minimal or negligent impact on the overall performance and if a middleware that solely optimizes latency is used, clear opportunities for achieving higher performance through alternative mechanisms are missed. It is subject of further studies to show the impact of these mechanisms on performance.

MPI/Pro, an MPI implementation from MPI Software Technology, Inc, is selected for this study. MPI/Pro offers both modes of message completion through a run-time choice. All other architectural characteristics of the two systems are the same. This allows for a fair comparison of the completion modes and their impact on performance.

6 MPI/Pro Architecture

MPI/Pro is a high-performance, multi-device MPI implementation for clusters of Linux and Windows workstations with user-level thread safety [5]. MPI/Pro provides specific optimizations for high-speed networks such as Giganet and Myrinet. The discussion in this paper is concentrated on Giganet. This network provides both synchronous and asynchronous methods for completion of send and receive operations. MPI/Pro propagates this capability to user processes by allowing them to specify which mode of completion they prefer. A run-time flag controls the choice between the two modes.

The asynchronous mode of MPI/Pro uses a device thread that blocks on events associated with incoming messages. This minimizes CPU time spent on communication and also provides for a greater degree of asynchrony between the communicating processes.

For transmitting long messages, MPI/Pro uses the remote DMA (RDMA) mechanism of Giganet's interface controllers. This reduces the CPU involvement in communication activities. The CPU utilization for messages larger than 32 kilobytes is in the range of 5-6%. The rest of CPU's time is available for computation. In addition to asynchronous notification, the device thread is also used to perform independent progress of long messages. This ensures timely delivery and results in a high utilization of the physical bandwidth. The asynchronous mode provides a clear opportunity of overlapping of communication and computation. While the network controller is processing a message, the user thread can perform useful computation. When the controller finishes the operation, it interrupts the CPU and then the control is transferred to the device thread that completes the transaction and propagates it to the user thread.

The synchronous mode of MPI/Pro eliminates the device thread from the reception of short messages. When a process expects a message, it can poll for its arrival. This excludes the overhead associated with the device thread context switch from the processing of incoming messages. If the communicating processes are tightly synchronized, as in the ping-pong tests, the polling mode can significantly reduce the round-trip time, at the expense of increased CPU usage. As opposed to most of the existing MPICH-based MPI implementations that use exclusively polling, MPI/Pro continues to use the device thread even in the synchronous mode of completion. The device thread is used for progress of long messages and also for handling flow control traffic. Thus, MPI/Pro eliminates the deficiency of the MPI implementations with pure polling architecture to require frequent calls to the library's progress engine.

7 Methodology

The goal of the experiments presented here is to establish a relationship or a lack thereof between point-to-point latency measured by ping-pong tests and overall applications performance represented by the NAS Parallel Benchmarks. The relationship is revealed through a comparative analysis. Two systems with different point-to-point latency attributes are studied. The difference in latency is related only to the method of MPI message completion – synchronous and asynchronous. All other components of the two systems are the same. In fact, the same message-passing middleware – MPI/Pro is used in the two systems – once in blocking and once in polling mode. The two systems are subjected first to point-to-point experiments for measuring latency and bandwidth. Ping-pong and streaming tests are used. Then, the NAS benchmarks are executed and their results are compared and related to the results from the point-to-

point measurements. Because the difference between the two systems is only in their point-to-point latency, the difference in the NAS benchmarks runs is attributed to the impact of message-passing latency, and hence to the mode of message completion.

8 Experimental Results

The experiments were carried out on clusters of Windows NT workstations. One of the clusters was AC3 Velocity at the Theory Center at Cornell University. The other cluster is deployed at the offices of MPI Software Technology. Both clusters are interconnected with Giganet cLAN. The AC3 nodes are quad PowerEdge DELL servers with Intel Xeon 500 MHz processors and 4 GB of RAM. The nodes of the second cluster are SAG “white box” computers equipped with a single Pentium II processor at 350 MHz and 288 MB of RAM. MPI/Pro with switching between polling and blocking was used for providing message passing. All experimental results are averaged over three or more measurements. In future efforts, experiments on Linux clusters interconnected with Giganet and other high-speed networks will be conducted.

8.1 Point-to-Point Performance

Figures 1 through 4 present the results from the tests for measuring roundtrip latency (L_{rtt}), one-way latency (L_{owl}), and round-trip based bandwidth (B_{rtt}) as described in section 4. Each graph presents numbers from experiments in both blocking and polling modes. Figure 1 demonstrates that the ping-pong latency of the polling completion mode is more than two times better than the latency of the blocking mode on the SAG cluster. The zero-length latency in polling mode is 19 microseconds while in blocking mode it is 43 microseconds. The difference can be attributed to the thread context switch that the asynchronous completion mode of MPI/Pro imposes on each

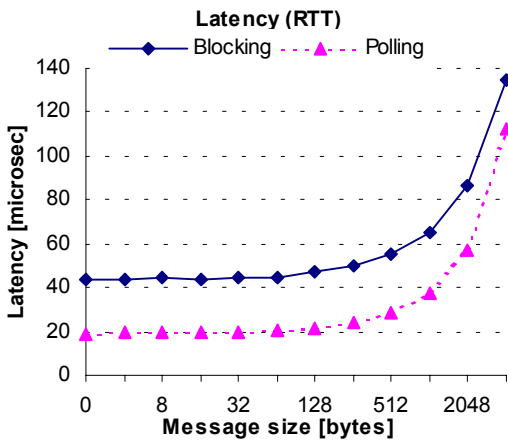


Figure 1. RTT latency on SAG cluster.

incoming message. Also, Giganet cLAN driver adds about 15 microseconds overhead for completing a

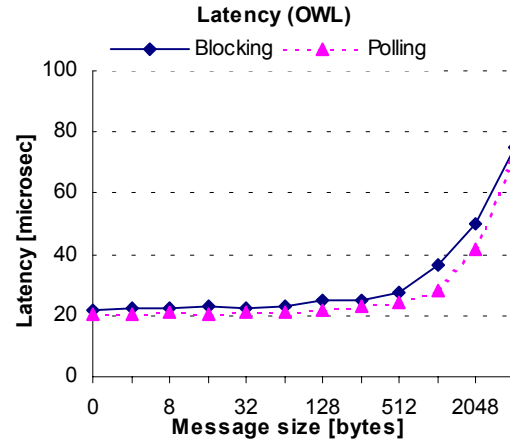


Figure 2. One-way latency on SAG cluster.

message in a blocking mode, which is caused by the interrupt handler. A detailed breakdown of the latency components of MPI/Pro using blocking mode can be found in [12].

An interesting observation can be made on Figure 2, which represents one-way latency measured with a

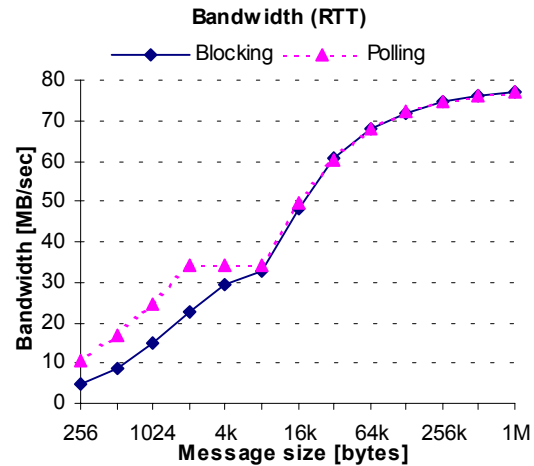


Figure 3. Point-to-point bandwidth on SAG cluster

streaming test. While the polling mode does not show any improvement in comparison to ping-pong, the blocking mode performs almost as well as the polling mode – reduction of more than two times. This shows that pipelining messages can hide the overhead associated with blocking mode. As was mentioned earlier, realistic communication patterns of applications lie in the space defined by ping-pong and streaming and are a combination of them. So, applications will not actually see the difference in overhead of the two modes as measured by the ping-pong test.

The bandwidth curves on Figures 3 and 4 show that for message sizes above 4 kilobytes the two completion modes are equivalent. The difference in short-message bandwidth is based on the fact that in this size range the round-trip time is dominated by the latency and not by the time for actual message transmission across the network. Typical numerical algorithms using large data sets and exchange predominantly messages with sizes greater than 4 kilobytes. Evidently, applications using these algorithms will not benefit from the higher bandwidth for short messages resulting from the lower latency of polling.

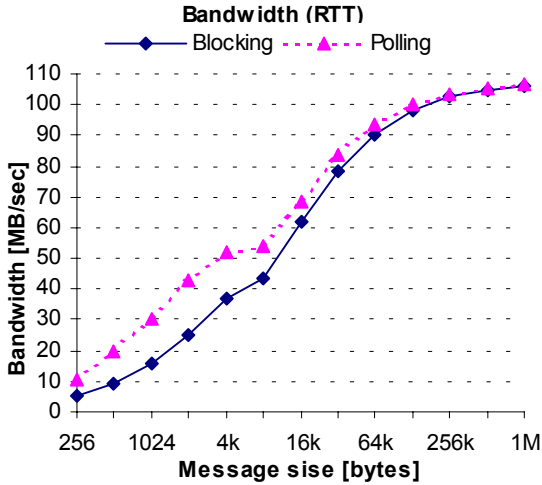


Figure 4. Point-to-point bandwidth on DELL cluster.

Polling bandwidth curves have a plateau around message sizes of 4 kilobytes. This plateau is caused by the increasing overhead of the extra copy at the receiver process used by the eager protocol of MPI/Pro. For larger sizes, the rendezvous protocol eliminates the extra copy and the curve rises again afterwards. If the protocol switchover size is less than 4 kilobytes, the overhead of the extra messages caused by the rendezvous protocol causes decline in polling bandwidth. The switchover size of MPI/Pro is chosen to offer an optimal balance for transmission of both short and long messages.

8.2 NAS Benchmarks Performance

NAS Parallel Benchmarks are used to demonstrate how the difference in latency of the two completion modes affects real applications. NAS benchmarks are a well-established collection of applications and kernel benchmarks for testing performance and scalability of parallel systems. NAS benchmarks are based on medium- to coarse-grain data parallel algorithms implemented with MPI. They have been originally written for MPP systems and naturally reflect the specifics of the MPI implementations for

these systems. Specifically, NAS benchmarks rely on high degree of synchrony among the computing processes, use primarily blocking and non-persistent mode of communication, and do not make use of overlapping and temporal locality.

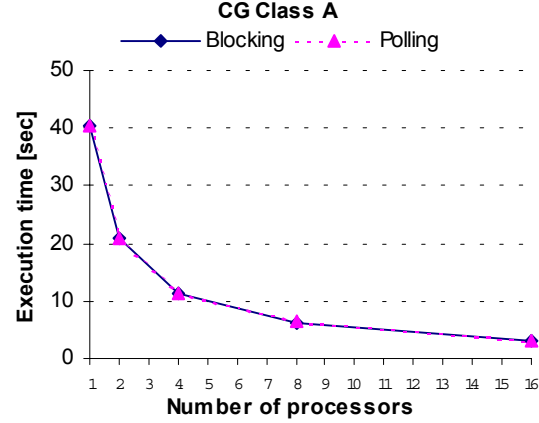


Figure 5. CG class A on DELL cluster.

The NAS CG and IS benchmarks were chosen for the experiments, both with class A data sets. The results on the DELL cluster are shown in Figures 5 and 6. (The results for the SAG cluster are similar and can be found on MPI Software Technology's web site.) The performance metric chosen for presenting the cluster performance is parallel execution time. It can be seen from the figures that the polling and blocking performance curves differ with quantities smaller than the standard deviation and they almost completely overlap.

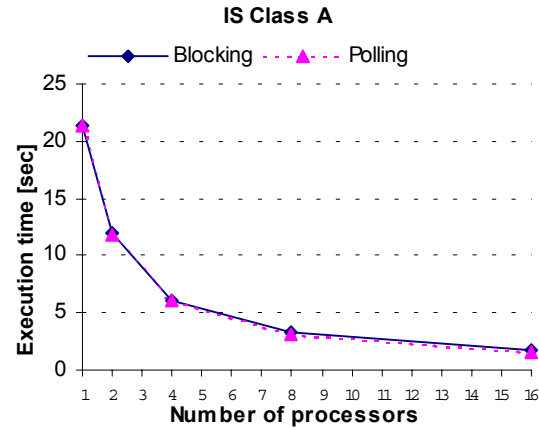


Figure 6. IS class A on DELL cluster

A conclusion can be drawn that the difference in latency of the two systems under investigation has no impact on the overall applications performance. This fact can be explained with the communication pattern and message sizes of the two benchmarks presented here. Figures 7 and 8 show the number and size of

Processors	2	4	8	16
send/recv	104x28k	104x28k	156x14k	156x14k
send/recv	104x4B	104x4B	208x4B	208x4B

Figure 7. Communication pattern of CG innermost loop. The loop is executed 15 times.

messages exchanged in the innermost loops of CG and IS respectively. It is clear that the bulk of data is transmitted by relatively few medium or large size messages. Consequently, CG and IS are bandwidth dependent, not latency dependent.

Processors	2	4	8	16
allreduce	4 kB	4 kB	4 kB	4 kB
alltoall	4 B	4 B	4 B	4 B
alltoallv	8 MB	2 MB	512 kB	128 kB

Figure 8. Communication pattern of IS innermost loop. The loop is executed 10 times.

9 Conclusions

At present, clusters of workstations interconnected with high-speed networks are becoming the predominant choice for building parallel systems. Often, their performance evaluation is based on limited scope ping-pong tests that measure point-to-point latency and bandwidth. Most message-passing systems that use zero-copy transfers reach the peak link bandwidth. Consequently, the evaluation procedure is further reduced to comparison of latency. In addition to the capabilities of network hardware, latency is significantly affected by the method of message completion. The synchronous method uses polling and achieves low latency at the expense of CPU overhead. The asynchronous method uses blocking system calls and threads or callbacks. Blocking leads to higher latency, but on the other hand, it reduces CPU time spent on communication and allows for overlapping of computation and communication.

The experiments presented in this paper show that the lower latency of polling in comparison to blocking does not lead to an improvement of the overall performance of medium to coarse grain data-parallel applications represented by the NAS benchmarks. The algorithms presented in these benchmarks exchange a small number of large-size messages. Therefore, the communication bandwidth has a stronger impact on the overall performance than latency. Although systems that use asynchronous completion have higher latency than systems with synchronous completion, they impose only minimal or no loss application performance. Polling used in synchronous completion systems results in tight synchronization between the communicating processes and uses CPU cycles for communication. As opposed to that, asynchronous completion offers a number of alternative mechanisms

for improving performance, such as overlapping of computation and communication, independent message progress, high sustainable bandwidth, optimized collective operations, and asynchronous processing. Medium to coarse grain data-parallel algorithms with regular communication pattern can significantly benefit from these mechanisms. Future work will demonstrate how applications that use these mechanisms achieve higher performance than applications that do not use them, even though the latter may be run on message-passing systems with lower point-to-point latency.

References

- [1] D. Bailey, E. Barszcz, J. Barton, D. Browning, R. Carter, L. Darum, R. Fatoohi, P. Frederickson, T. Lasinski, R. Schreiber, H. Simon, and V. Venkatakrisham. The NAS Parallel Benchmarks, *International Journal of Supercomputer Applications*, 5 (3): 63-73, 1991.
- [2] N. Boden, D. Cohen, R. Felderman, A. Kulawik, C. Seitz, J. Seizovic, and W. Su. Myrinet: A Gigabit-per-second Local Area Network. *IEEE Micro*, 15 (1): 29-36, February 1995.
- [3] Compaq, Intel, and Microsoft. Virtual Interface Architecture Interface Specification, Version 1.0. December 1997, <http://www.viarch.org>.
- [4] D. Culler, R. Karp, D. Patterson, A. Sahay, K. Schauser, E. Santos, R. Subramonian, and T. Eicken. LogP: Towards a Realistic Model of Parallel Computation. In *Proc. Of the 4th ACM Symp. On Principles and Practice of Parallel Programming*: 1-12, San Diego, California, May 1993.
- [5] R. Dimitrov and A. Skjellum. Efficient MPI for Virtual Interface (VI) Architecture. In *Proc. of the 1999 Int. Conf. on Parallel and Distributed Processing Techniques and Applications*, vol. 6: 3094-3100, Las Vegas, Nevada, 1999.
- [6] Giganet, Inc. Giganet cLAN Family of Products, <http://www.giganet.com/products>, 1999.
- [7] W. Gropp, E. Lusk, N. Doss, and A. Skjellum. A High-performance, Portable Implementation of the MPI Message Passing Interface Standard. *Parallel Computing*, 22 (6): 789-828, September 1996.
- [8] A. Gupta and V. Kumar. Performance Properties of Large Scale Parallel Systems, *Journal of Parallel and Distributed Computing*, 19: 234-244, 1999.
- [9] Message Passing Interface Forum. MPI: A Message-Passing Interface Standard. *International Journal of Supercomputer Applications*, 8 (3/4): 165-414, 1994.
- [10] L. Valiant. A Bridgin Model for Parallel Computation, *Communications of the ACM*, 33 (8):103-111, August 1990.
- [11] D. Eager, J. Zahorjan, and E. Lazowska. Speedup versus Efficiency in Parallel Systems, *IEEE Transactions on Computers*, 38 (3): 408-423, 1989.
- [12] R. Dimitrov and A. Skjellum. An Efficient MPI Implementation for collective Virtual Interface Architecture – Enabled Cluster Computing, In *Proc. of the Third MPI Developer's and User's Conf.*: 15-24, Atlanta, Georgia, 1999.